

Apache Airflow

Apache Airflow is a tool used for automation, where tasks and workflows run automatically at scheduled times

Note : In Apache Airflow, DAGs are usually created manually by developers using Python code

Example:

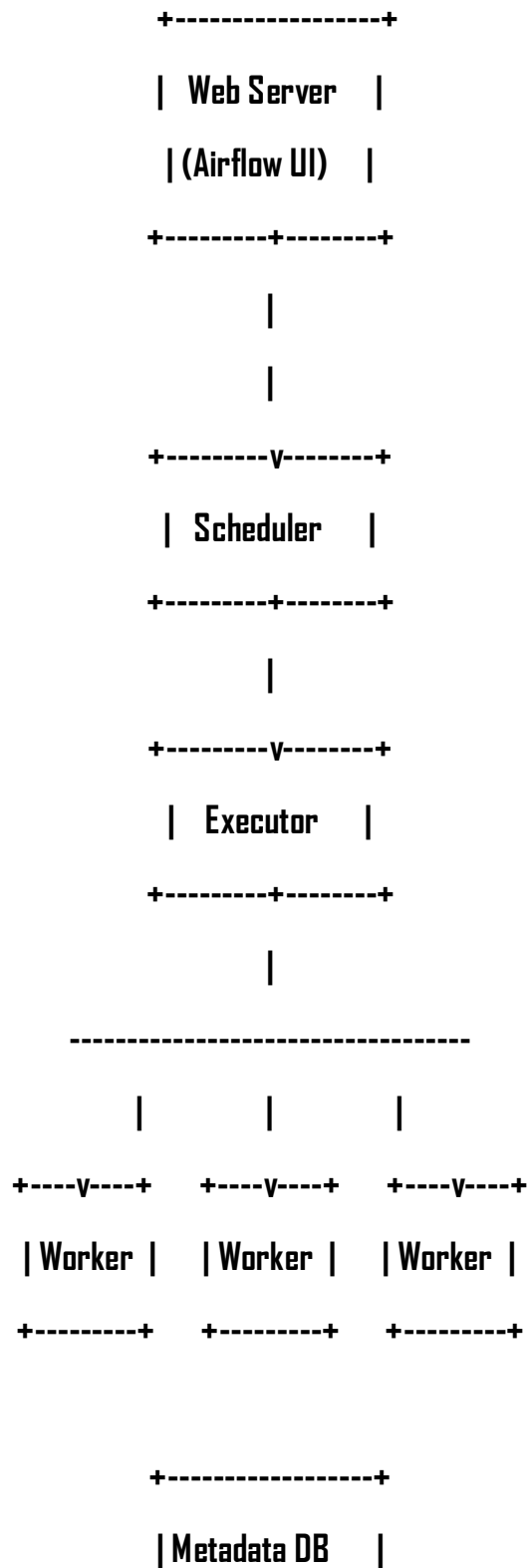
Suppose every day you need to:

1. Read data from MySQL
2. Clean data
3. Store data in Hadoop
4. Generate reports
5. Send email notification

Instead of doing manually, Airflow automates the entire process.

Run multiple tasks in the correct order automatically at scheduled times

Airflow Architecture



What is DAG in Apache Airflow?

DAG stands for:

D → **Directed**

A → **Acyclic**

G → **Graph**

It is the core concept of Airflow.

A DAG is:

A workflow execution plan that defines tasks and their execution order.

Simple Meaning

Suppose you have 3 tasks:

Task A → Task B → Task C

Meaning:

- First run Task A
- Then Task B
- Then Task C

This workflow is called a DAG.

Why Directed?

Because tasks have direction/order.

Example:

A → B A runs before B.

Why Acyclic?

No loops allowed.

✗ Wrong:

$A \rightarrow B \rightarrow C \rightarrow A$

This creates infinite loop. (This won't happen in DAG)

Why Graph?

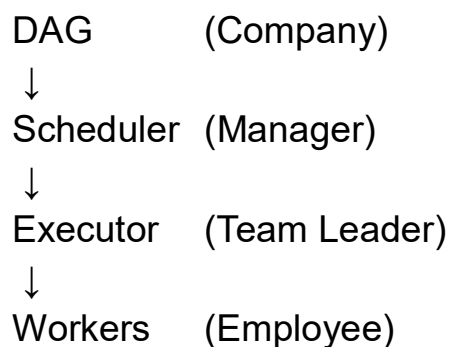
Because tasks are connected like a graph/network.

Airflow Executor

In Apache Airflow:

Executor is a component that decides HOW and WHERE tasks should run and It does NOT process big data itself.

Airflow Architecture



DAG → Defines the workflow, execution plan, task dependencies, and scheduling information.

Scheduler → Decides what task should run and when to run based on the DAG.

Executor → Assigns/distributes tasks to workers for execution.

Workers → Execute the actual tasks.

Airflow Executor Responsibilities

Airflow executor:

- Receives tasks from scheduler
- Assigns tasks to workers
- Handles parallel execution
- Manages task execution strategy

Important Point

Airflow executor mainly MANAGES execution.

Workers actually execute tasks.

Advantages of AirFlow

- Automation of tasks and workflows.
- Automatic scheduling of jobs.
- Follows acyclic workflow (no infinite loops).
- Maintains proper workflow execution.
- Handles task order and dependencies.
- Reduces manual work.
- Supports monitoring through UI.
- Retries failed tasks automatically.
- Supports parallel execution.
- Easy integration with Spark, Hadoop, AWS, etc.
- Scalable for large data pipelines.
- Open-source and free to use.